Interview Prep — Agent Routing & Scope

> *Study material for the pms-semantic-layer project. Continues 01-dbt-semantic-layer-governance.md; continues in 03-live-agent.md.*

Continues the format and numbering from 01-dbt-semantic-layer-governance.md (sections 1-7: docs blocks, schema separation, exposures, contracts, freshness). This file covers the routing/scope guide built for the future text-to-SQL agent (chapter 12-13).

---

8. Routing vs scope -- two different questions, two different files

Q: `agent_routing.yml` and `agent_scope.yml` sound similar. What's the actual distinction?

- **Weak answer**: "One's for routing, one's for permissions." (Restates the names, doesn't explain the underlying split.)
- **The gap**: they answer two unrelated questions about the same incoming request. Routing answers *"given this business question, which model(s) should I query and how do they join?"* -- a correctness/efficiency concern. Scope answers *"regardless of the question, what am I never allowed to touch?"* -- a safety/access-control concern that holds even for a perfectly-routed query.
- **Correct answer**: routing is about getting to the right data; scope is a hard boundary independent of routing -- read-only enforcement (already built in chapter 07's `is_read_only`), schema-level restriction (`marts` only, never `staging` or raw `public`), and column-level PII restriction (`guest_name`, `email`, `phone` are never surfaced even from an otherwise-permitted table).
- **Study**: this maps directly onto a real access-control pattern -- routing is closer to *query planning*, scope is closer to *authorization*. They compose (a request must pass both), but conflating them into one file would make it harder to reason about which one caught a given failure.

---

9. Filling in a `question_types` entry in `agent_routing.yml`

Q: What's the actual checklist for writing one entry in `agent_routing.yml`, given a new business question?

- **Weak answer**: "Write the question type and which table to use."
- **The gap**: "which table" is underspecified the moment a question needs more than one table -- and deciding *which* column-level join key is correct requires knowing the semantic difference between similarly-named columns across models (this project has more than one `property_manager_id`, for exactly this reason).
- **Correct checklist**, applied in this chapter:
  1. **`type`**: a short, stable identifier for the question shape (not the literal question text) -- e.g. `revenue_per_property_manager`, not "how much did property manager X earn."
  2. **`keywords`**: terms likely to appear in a real user question, used for matching intent -- include synonyms (`revenue`, `earnings`, `RevPAN`) since users won't phrase things consistently.
  3. **`primary_mart`**: the mart that answers most of the question directly -- prefer the most-derived mart that already has the KPI (`mart_kpi` over `mart_aggregates` if the KPI already exists there, to avoid re-deriving a ratio the mart layer already computed and validated).
  4. **`secondary_tables` + `join_keys`**: only when the primary mart's grain is missing something the question needs (e.g. `mart_kpi`'s grain is `accommodation_id, month` --

it has no `property_manager_id` at all, so a property-manager-scoped question needs a join).

5. **`notes`** : the non-obvious caveat a routing table alone can't express -- most importantly, *which* semantically-similar column to use when more than one candidate exists.

- **Study**: re-read `_sources.yml` 's existing note on `reservation.property_manager_id` (chapter 08) -- it already documents the exact distinction this routing guide has to operationalize.

Q: This project has `accommodation.property_manager_id` and `reservation.property_manager_id` . Why would an agent ever need to be told which one to use, instead of just picking either?

- **Weak answer**: "They're basically the same, either works most of the time."
- **The gap**: "most of the time" is exactly the failure mode this whole project has been defending against since chapter 07 -- a query that's *usually* right is indistinguishable from a query that's *always* right until the one case where an accommodation was reassigned to a different property manager, at which point they diverge and the wrong choice gives a plausible, wrong number silently.
- **Correct answer**: `accommodation.property_manager_id` reflects the *current* manager (join via `stg_accommodation` when the question is "who manages this now, and what's their portfolio doing"). `reservation.property_manager_id` is a *frozen snapshot* at booking time (join via `stg_reservation` when the question is historical -- "who was managing this at the time of that booking"). The routing guide's `notes` field exists specifically to encode this decision so the agent doesn't have to (re-)discover it per query.
- **Study**: this is the same category of bug as `booked_at` / `listed_at` / `updated_at` from chapters 09-11 -- a project with several near-synonymous columns that differ only in *when* they're true. The general interview-worthy pattern: whenever a schema has two columns that sound interchangeable, ask "is one of these a live value and the other a snapshot?" before assuming they're redundant.

---

10. The fallback case: what happens when nothing matches

Q: `agent_routing.yml` has 8 `question_types` with keyword lists. What should the agent do if a real user question doesn't match any of them?

- **Weak answer**: "Fall back to its general knowledge and try to answer anyway." (This is, verbatim, the exact failure mode this entire project exists to defend against -- an LLM producing a plausible answer with no grounding in the real schema, chapter 07's silent "occupancy rate: 70%" hallucination.)
- **The gap**: a routing guide that only defines the *matched* cases and says nothing about the *unmatched* case is incomplete by construction -- it silently assumes 100% keyword coverage, which is never true in practice, and leaves the highest-risk case (an unrecognized question) completely unspecified.
- **Correct answer**: a routing guide needs an explicit `fallback` behavior as a first-class citizen of the config, not an afterthought -- in this project, `fallback.action: ask_clarification` , with an explicit `never: guess_or_query_without_a_matched_type` and a concrete message template the agent can surface to the user. The agent asking "I don't have a rule for this, can you rephrase?" is a *feature*, not a failure -- it's the same principle as `NULLIF` returning NULL instead of a fake 0 (chapter 10): an honest "I don't know" beats a confident wrong answer every time.
- **The rule**: any routing/classification system with a fixed set of known categories needs to treat "none of the above" as a real category with real, specified behavior -- never leave it implicit.

11. Reviewing a routing guide as a domain expert, not just for YAML correctness

Q: A routing config with valid YAML and sensible-looking entries passed review. What else is there to check?

- **Weak answer**: "Nothing, if it parses and each entry has the right fields." (Confuses syntactic validity with semantic correctness -- the more dangerous kind of bug in a routing table, since it fails silently by routing to a *plausible but wrong* place, not by erroring.)
- **The gap**: an expert review of a routing table should look for things a YAML linter can't catch:
  1. **Missing common cases** -- does every entry require a join that a simpler, more common phrasing of the same question wouldn't need? (Found here: no plain `revenue` type existed without a forced property-manager join.)
  2. **Keyword ambiguity between entries** -- can one real question's wording match more than one `type`, with no tie-breaking rule?
  3. **Grain mismatches hiding in a join** -- does a `join_keys` entry silently combine two tables with different natural grains (e.g. a monthly mart joined to a table with no time dimension at all), producing numbers that are technically computable but analytically misleading?
- **Study**: this is the same discipline as reviewing a dbt model for a business-logic bug (chapters 08-11) applied one layer up, to the config that tells an agent *how* to query the models -- the review target changed, the failure mode (a query that "works" but answers the wrong question) didn't.

Four concrete findings from this review, and how each was resolved:

1. **Missing generic case forcing an unnecessary join.** `revenue` / `adr_revpan` questions with no property-manager scope were being forced through `revenue_per_property_manager`'s join to `stg_accommodation`. Fix: added standalone `revenue` (from `mart_aggregates.revenue`) and `adr_revpan` (from `mart_kpi`) types with no joins, and trimmed the specific type's keywords down to only what makes it specific (`"property manager"`, not the base metric words it shares with the generic type).
2. **No fallback for unmatched questions.** Added a top-level `fallback` block (`action: ask_clarification`, explicit `never: guess_or_query_without_a_matched_type`) -- see section 10.
3. **Keyword ambiguity, two forms.** (a) Generic vs. specific overlap (`"revenue"` in both a generic and a scoped type) -- resolved with a `matching_rules.on_multiple_matches: prefer_the_more_specific_type` policy, plus removing the duplicated generic word from the specific type's own keyword list. (b) An overly broad single-word keyword (`"accommodation"` in `accommodation_count`) matching almost any question that names a specific accommodation -- resolved by requiring full intent phrases (`"how many accommodations"`) instead of a bare noun.
4. **Grain mismatch inside a join** (`adr_vs_cost`): `stg_accommodation_cost` has no time dimension, so joining it to a monthly mart and comparing a single month's ADR against a lifetime cost figure is misleading even without a fan-out bug. Fix: pre-aggregate both sides to the same grain (SUM per `accommodation_id`, lifetime) before joining -- compare total revenue to total cost, not one month's ratio to an all-time total.

Q: Why fix #1 and #3 in the same pass instead of one at a time?

- **The gap worth noting**: fixing #1 (adding a generic `revenue` / `adr_revpan` type) *introduced* a fresh instance of #3 (a newly duplicated `"revenue"` keyword) immediately. This is a realistic pattern, not a coincidence -- adding a new category to a keyword-matched system is exactly the

moment new ambiguity gets introduced, since the new entry's natural keywords overlap with whatever it's meant to be more specific than. Any time a routing/classification config grows a new entry, re-checking for fresh ambiguity against *existing* entries has to be part of that same change, not a separate later pass.

12. Domain-expertise findings, not just config-mechanics findings

Q: A routing config can be internally consistent (no YAML errors, no keyword ambiguity, every join grain-matched) and still be missing something important. What kind of gap does that describe?

- **Weak answer**: "Missing test coverage." (That's a mechanical framing -- these gaps aren't caught by any test, because nothing about them is *wrong*, they're *incomplete* relative to what the business domain actually asks.)
- **The gap**: a config can pass every structural check and still fail a domain-expertise review, because that review asks a different question: not "is this correct," but "does this match how someone who actually runs vacation rentals would ask about their business."
- **Two real findings from this project's final pass**:
  1. **A keyword promised a normalization the logic didn't deliver.** `adr_vs_cost` had a `"cost per night"` keyword, but its notes only computed a lifetime cost *total* -- never divided by anything. A true cost-per-night figure needs the same normalization RevPAN already uses on the revenue side: `SUM(cost) / SUM(available_nights)`, grouped by `accommodation_id`, so an accommodation listed for 3 years isn't unfairly compared against one listed for 3 months on raw totals alone.
  2. **Two fully-documented, real dimensions had zero routing coverage.** `channel` (AIRBNB/BOOKING/EXPEDIA) and `season` (HIGH/LOW) are core columns on `reservation`, explicitly designed and backfilled earlier in this project (chapter 08-09) -- and "which channel brings in the most revenue" / "how does occupancy differ by season" are exactly the kind of question a real property manager asks constantly. Neither had a `question_types` entry, so both would have silently gone to the (safe, but still incomplete) fallback.
- **The rule**: a routing config's coverage should be checked against the *schema's own documented dimensions* (every column with an `accepted_values` test is a candidate "breakdown by X" question), not just against the specific example questions used to originally scope the file (chapter 07's 6 questions, in this project's case). A dimension worth a CHECK constraint and a dbt test is usually also a dimension worth a routing entry.

---

13. When "marts only" access control collides with real routing needs

Q: `agent_scope.yml` says the agent can only read from the `marts` schema. Roughly half of `agent_routing.yml`'s entries pointed straight at `stg_reservation` / `stg_accommodation`. What does that mean?

- **Weak answer**: "One of the two files needs a small tweak."
- **The gap**: this isn't a small inconsistency, it's a design contradiction discovered by writing the second config against the first -- half the routing table's entries described queries the scope config would then refuse to let the agent run at all. Writing scope *after* routing, and actually checking one against the other, is what surfaced it; writing them independently (or writing scope first and assuming routing would "just comply") would have shipped a broken agent.
- **Correct answer, and what "mart" actually means**: the fix wasn't to relax the schema restriction, it was to recognize that "marts only" doesn't mean "aggregates only" -- a mart can be a governed,

tested, documented *fact* or *dimension* table too, not just a KPI rollup. Built two: `fct_reservation` (one row per reservation, PII-scrubbed -- excludes `guest_name`) and `dim_accommodation` (one row per accommodation, flattening `occupancy_rules` 1:1 and pre-aggregating `accommodation_cost` so no consumer ever joins the raw multi-row cost table directly). Every routing entry that referenced staging was repointed at one of these instead.
- **Study**: this is the standard warehouse layering pattern -- `staging` (uncurated, internal, no guarantees) → `marts` (curated, tested, access-controlled; both fact/dimension AND aggregate models belong here) → consumers (BI, agents) that only ever see the last layer. Confusing "mart" with "only pre-aggregated KPI table" is a common shorthand that breaks exactly this way once a consumer needs row-level or dimensional data it can't get from an aggregate.

Q: `dim_accommodation` pre-aggregates `accommodation_cost` (SUM(amount) GROUP BY accommodation_id) inside the mart itself, rather than leaving that to query time. Why is that a better place for it than leaving the aggregation to whoever queries the mart?

- **Weak answer**: "It's more convenient."
- **The gap**: convenience isn't the point -- safety is. `agent_routing.yml` 's `adr_vs_cost` entry had already documented a real fan-out risk from joining the raw, multi-row cost table directly to a KPI mart. Pre-aggregating inside `dim_accommodation` doesn't just make queries shorter, it makes the fan-out bug *structurally impossible* for any consumer of this mart -- the dangerous join now happens once, inside a tested dbt model, instead of being re-derived (and potentially re-broken) by every future query against it, whether written by a human or an agent.
- **The rule**: when a raw table's grain doesn't match what most consumers need (1:many where 1:1 is wanted), collapsing it inside the mart -- not documenting the collapse as a caveat for every future query -- removes an entire class of bug at its root, the same principle behind fixing the `listed_at` bug in the data generator (chapter 11) instead of the mart.

14. Defense-in-depth for a rule that's already true

Q: `agent_scope.yml` declares `forbidden_columns` (guest_name, email, phone, first_name, last_name) even though no current mart exposes any of them. Isn't that dead configuration?

- **Weak answer**: "Yes, it does nothing right now, but it's harmless to leave in."
- **The gap**: "does nothing right now" undersells its purpose -- a rule like this isn't there to change today's behavior, it's there to fail loudly the day someone (a future chapter, a future contributor) adds a new mart that accidentally includes `guest_name` for convenience. Without the explicit list, that mistake would silently become "allowed" the moment it happened, since nothing would flag it.
- **Correct answer**: this is defense-in-depth -- a second, independent layer of protection that doesn't rely on every future mart's author remembering the PII rule unprompted. The current marts satisfy it by design (not by coincidence), but the config's job is to keep satisfying it after the design intent is no longer fresh in anyone's memory.
- **Study**: the same reasoning applies to `max_rows: 500` -- no query in the current `agent_routing.yml` needs more than ~613 rows, so the limit doesn't bind today, but it exists to catch an unscoped future query before it does something expensive or nonsensical, not to constrain anything currently working. Explicitly justified against real numbers (613 accommodations, 5346 reservations) and the precedent already set by chapter 07's `fetchmany(6)`, scaled up for a real analytics agent instead of a demo transcript.